To address these questions, this work aims to provide a structured reference for understanding the performance trade-offs and practical limitations of LM inference on resource-constrained devices. Beyond offering insights for hardware-software co-design in edge AI, our findings contribute to a broader understanding of how edge devices can (or cannot) handle LM workloads effectively. This awareness is relevant for scenarios where distributed edge nodes must collaborate to execute LM inference, helping to inform the design of decentralized AI systems. Additionally, this study provides different insights for AI researchers optimizing inference efficiency, system architects designing multi-node edge deployments, and industry practitioners integrating LMs into latency-sensitive IoT and 5G applications. The key contributions (C#) of this study are:

- **(C1)** Addressing **(RQ1-3)**, we benchmark 11 generative LMs on two widely used SBCs, analyzing key performance indicators such as memory usage, inference speed, and energy efficiency to quantify the feasibility of edge inference.
- **(C2)** Answering **(RQ2)**, we evaluate the effect of quantization and model scaling on inference efficiency, resource utilization, and model performance, highlighting the trade-offs between accuracy and computational cost.
- **(C3)** Directly addressing **(RQ3)**, we compare CPU-based inference against GPU acceleration, investigating their impact on execution speed, energy consumption, and practical deployment feasibility on edge devices.
- **(C4)** Responding to **(RQ4)**, we analyze the impact of power modes, threading configurations, and system settings, while also evaluating micro-architectural metrics, such as cache misses and context switches, to uncover bottlenecks and efficiency gaps in edge-based LM execution.
- **(C5)** Informed by **(RQ5)**, we assess the practical challenges of running LMs at the edge, including inference cost analysis, qualitative benchmarking of the models, and aspects such as usability related to real-world applicability, providing a broader perspective beyond raw performance metrics.

The remainder of this paper is structured as follows: Section 2 provides background information on enabling technologies such as transformers and inference engines. Section 3 presents a selection of closely related work. Section 4 details the experimental methodology, while Section 5 presents the empirical results. In Section 6, these results are further discussed and contextualized, leading to the final conclusions in Section 7.

2 Background

This section provides an overview of the key concepts relevant to our study. We first describe the architecture of LMs, including the transformer architecture, key-value cache (KV-cache) utilization, and model compression techniques such as quantization. Then, we introduce inference backends commonly used to execute LM inference on edge devices, which play an important role in determining computational feasibility.

Transformer Architecture and Large Language Models. The transformer architecture [56] was a breakthrough discovery, and most state-of-the-art LMs are still based on it. Its success stems from the multi-head attention mechanism, which does not rely on convolutions or recurrences, unlike previously established architectures. The attention mechanism computes key, value, and query vectors, where the key and value vectors of each token can be cached in the KV-cache and reused in subsequent forward passes. A token is the fundamental

unit of text that language models process. Tokens can represent entire words, subwords, or even individual characters, depending on the tokenization method used. Tokenization is the process of dividing a given text into discrete units (tokens). The KV-cache stores key and value vectors for all tokens within the current context, where the context refers to the sequence of previous tokens that still influence the generation of the next token. The maximum number of tokens that can be stored in the context is referred to as the context size. Because of the KV-cache, the generation of the first token exhibits different computational characteristics compared to subsequent tokens. As a result, generative transformer inference is typically divided into two phases: the *prefill phase*, also called initialization or prompt processing phase, and the *generation phase*.

In the prefill phase, the model calculates the key and value vectors for every input token and then stores them in the KV-cache. This process is computationally expensive, making the prefill phase compute-bound. Conversely, in the generation phase, these cached vectors are reused, eliminating the need for recomputation. As a result, this phase is memory-bound, as it involves frequent memory accesses. Without a KV-cache, the time complexity of the attention layer is quadratic with respect to sequence length, whereas utilizing a KV-cache reduces the complexity to linear.

The original transformer architecture follows an encoder-decoder approach [9], consisting of two independent neural networks interacting with each other. The encoder computes a variable-length sequence and encodes it into a fixed-length vector. The decoder then takes this vector as input and generates a variable-length sequence as output. In the context of language processing, these variable-length sequences correspond to *tokenized* text sequences. Over time, new model variants emerged, using only encoders or only decoders. Encoder-only models tend to perform better for tasks that require language understanding (e.g., sentiment analysis), but they cannot generate text independently. A notable set of encoder-only models belongs to the BERT-family [13]. Decoder-only models are typically well-suited for tasks involving text generation. Examples of this model class include GPT-4 [41] or Llama 3 [19].

Due to the sheer size of LMs, various model compression techniques have been proposed to improve efficiency, one of the most widely used being quantization. Quantization reduces the model size by lowering the precision of model parameters [16, 26, 58]. Some quantization techniques focus solely on quantizing model weights, such as GPTQ [16] and k-quants [29]. Other approaches, such as SmoothQuant [58], also quantize model activations, further reducing computational complexity. Quantization techniques can further be classified into *post-training* quantization and *quantization-aware training*. Post-training quantization (e.g., GPTQ, k-quants, SmoothQuant) applies quantization after a model has been fully trained. Quantization-aware training (e.g., BitNet [31, 57]) incorporates quantization during training, reducing quality loss. However, it requires training from scratch, making it computationally expensive and inapplicable to pre-trained models. Some post-training quantization techniques embed limited re-training rounds to mitigate quality loss. These are referred to as n-shot quantization, where n denotes the number of re-training iterations. For example, GPTQ is a 1-shot quantization technique, while 0-shot quantization techniques (e.g., certain k-quants) require no re-training at all.

Inference Backends for LMs. An inference backend is a software tool required to execute AI model inference on a device. Various backends support LM inference across different hardware architectures, enabling execution on both CPU- and GPU-based systems. One of the most widely used inference backends for LMs is `llama.cpp` [17], known for its broad hardware support and ability to run models on both CPUs and GPUs.

This backend system was chosen for this work, because of its flexibility and compatibility with various LM architectures. Another popular backend is `MLC LLM` [33], which offers a similar range of hardware and model support as llama.cpp. Additionally, it supports various quantization schemes [29], making it well-suited for optimized on-device inference. `picoLLM` [47] is another alternative, providing wide hardware support but offering a more limited model selection than MLC LLM and llama.cpp. It supports quantization and claims that its X-bit quantization significantly outperforms GPTQ [16], which is one of the most widely used post-training quantization techniques. For on-device inference, another backend is `TinyChat` [23], which supports Activation-aware Weight Quantization (AWQ) [26] and SmoothQuant [58]. However, it currently supports fewer models than llama.cpp or MLC LLM and does not yet enable inference on mobile devices. Finally, Google's `MediaPipe LLM Inference API` [18] enables LM inference capabilities for mobile phones and web applications. However, it currently supports only a limited selection of models.

3 Related Work

Benchmarking the performance of deep neural network (DNN) models on devices is not a new topic, and several benchmark suites have been proposed over time, such as AI Benchmark [22], EmBench [5] and MLPerf [50]. Among these, MLPerf has emerged as an industry standard for quantifying device performance for DNN training and inference across different DNN tasks, such as image classification or text summarization.

Table 1. Comparison with closely related papers

		Present work	[24]	[36]	[25]	[14]	[52]
Device Types	SBC (CPU)	✓	×	✓	×	✓	✓
	SBC (GPU)	✓	✓	×	×	×	✓
	Mobile Phones	×	✓	×	✓	×	×
Metrics	Latency/Throughput	✓	✓	✓	✓	✓	✓
	Energy	✓	✓	×	×	×	✓
	Memory	✓	✓	✓	✓	✓	✓
	Micro-architectural Metrics	✓	×	×	×	✓	×
Model Architectures	encoder-only	×	×	×	×	×	✓
	decoder-only	✓	✓	✓	✓	✓	×
	encoder-decoder	✓	×	×	×	×	×
Discussion Points	Configurations	✓	(✓)	×	×	×	×
	Throughput-Energy Trade-off	✓	×	×	×	×	×
	Usability	✓	✓	×	×	×	×
	Cost Analysis	✓	×	×	×	×	×

Recent studies have investigated the performance of transformer-based LMs on edge devices. A comparative overview of these works is provided in Table 1. Sarkar et al. [52] analyzed the performance of BERT-family models [13] on multiple SBCs across various NLP tasks. Their work also examined the impact of pruning on performance and accuracy. In contrast, our study focuses on generative models rather than encoder-only architectures and employs quantization as the primary model compression technique. Other works that focus on generative transformer models include [25], which analyzes LM inference on mobile phones, differing from our focus on SBCs. [14], and [36], which evaluates LM inference on SBCs but lack energy consumption measurements and do not discuss usability considerations for LM deployment at the edge. The study most closely related to our work is MELTing Point [24], which also investigates LM inference feasibility on edge devices, analyzing computational performance and energy usage. However, their study primarily focuses on